

Computer Science

07. 메모리 관리

7주차: Memory Management

목차 INDEX

01 지난 이야기

- 운영체제 개요
- 컴퓨터구조
- 프로세스와 스레드
- CPU 스케줄링
- 프로세스 동기화
- Deadlock

02 메모리

- 메모리 주소
- 스택 / 힙
- 메모리 할당

03 Paging

- Fragmentation
- Compaction
- Paging

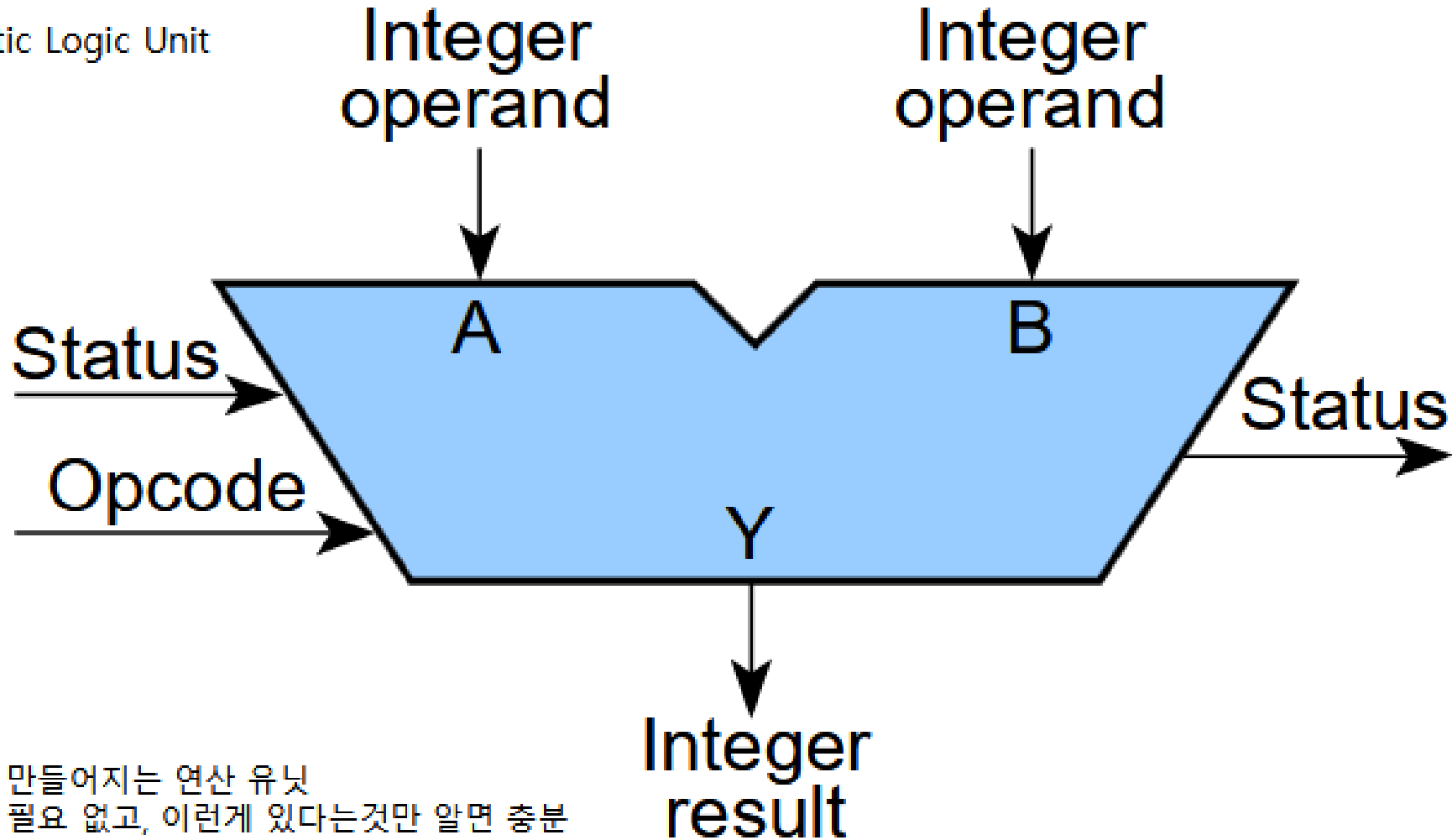
01

지난 이야기

01 ~ 06

01 컴퓨터 시스템과 운영체제 개요

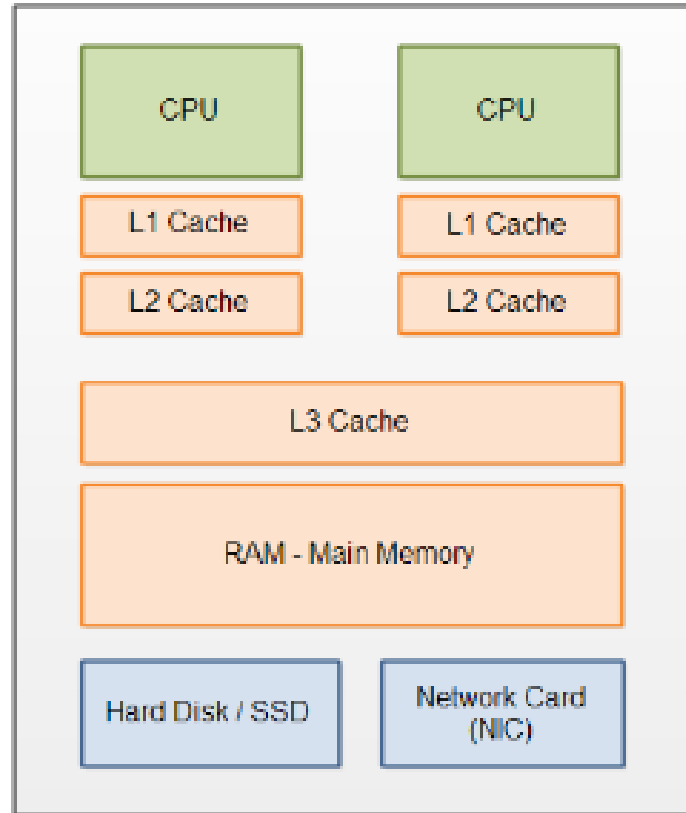
ALU Arithmetic Logic Unit



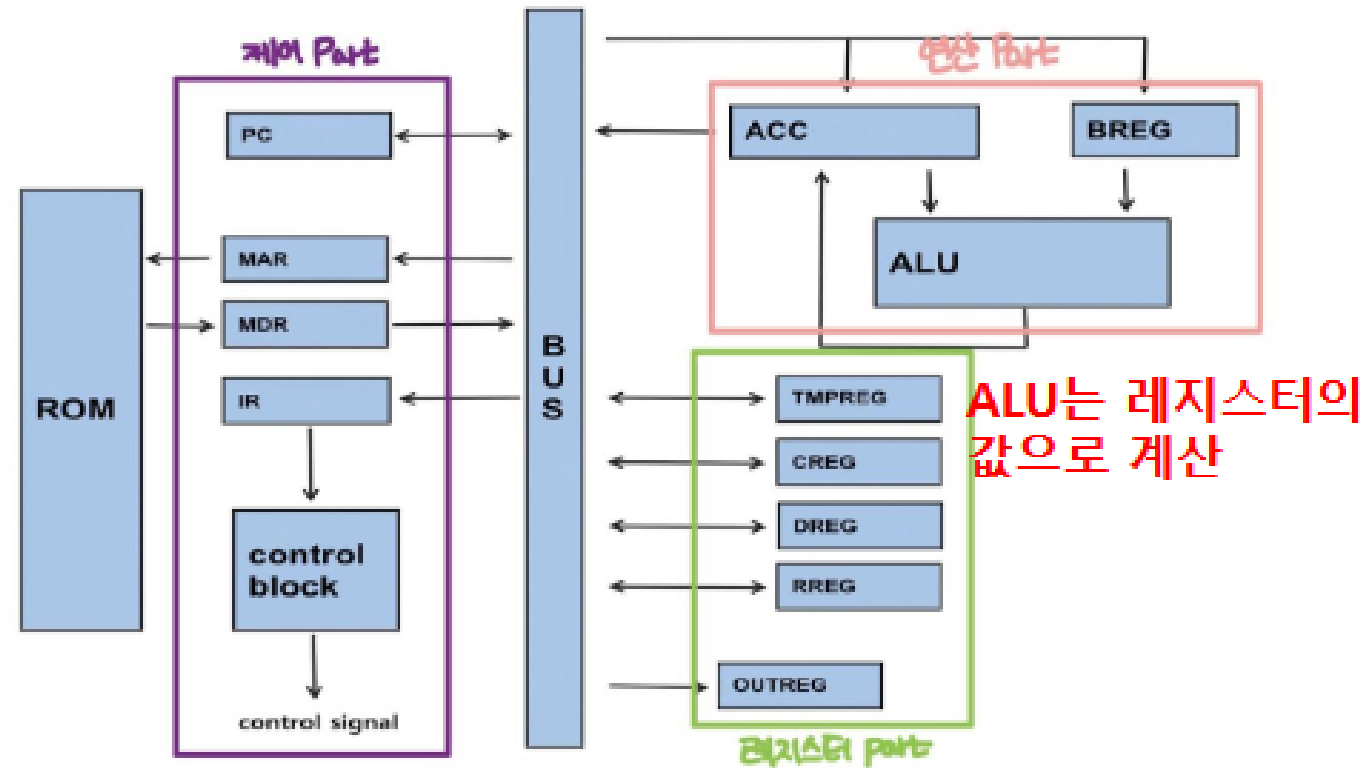
- 논리 게이트가 모여 만들어지는 연산 유닛
- 세부적인 내용은 알 필요 없고, 이렇게 있다는것만 알면 충분

01 컴퓨터 시스템과 운영체제 개요

Cache & Register



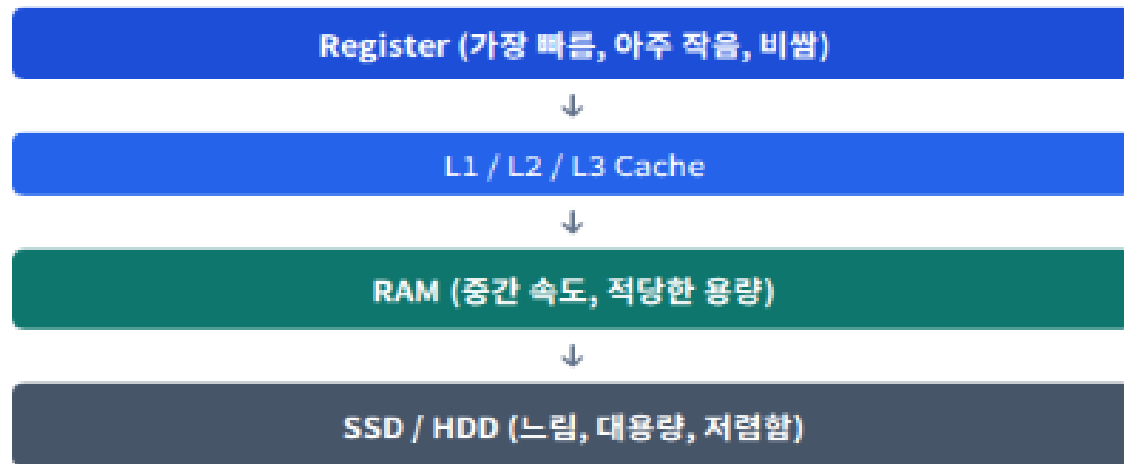
- CPU 칩 안에 들어있는 고속 임시 저장소
- RAM에서 가져온 데이터 임시 저장
- 빠른 CPU의 속도를 맞추기 위해 필요



- CPU 칩 안에 들어있는 초고속 임시 저장소
- 연산에 필요한 데이터와 주소 저장
- CPU가 명령 실행 및 연산 수행을 위해 사용

01 컴퓨터 시스템과 운영체제 개요

메모리 계층 구조 Memory Hierarchy



메모리 계층 구조의 핵심 특징

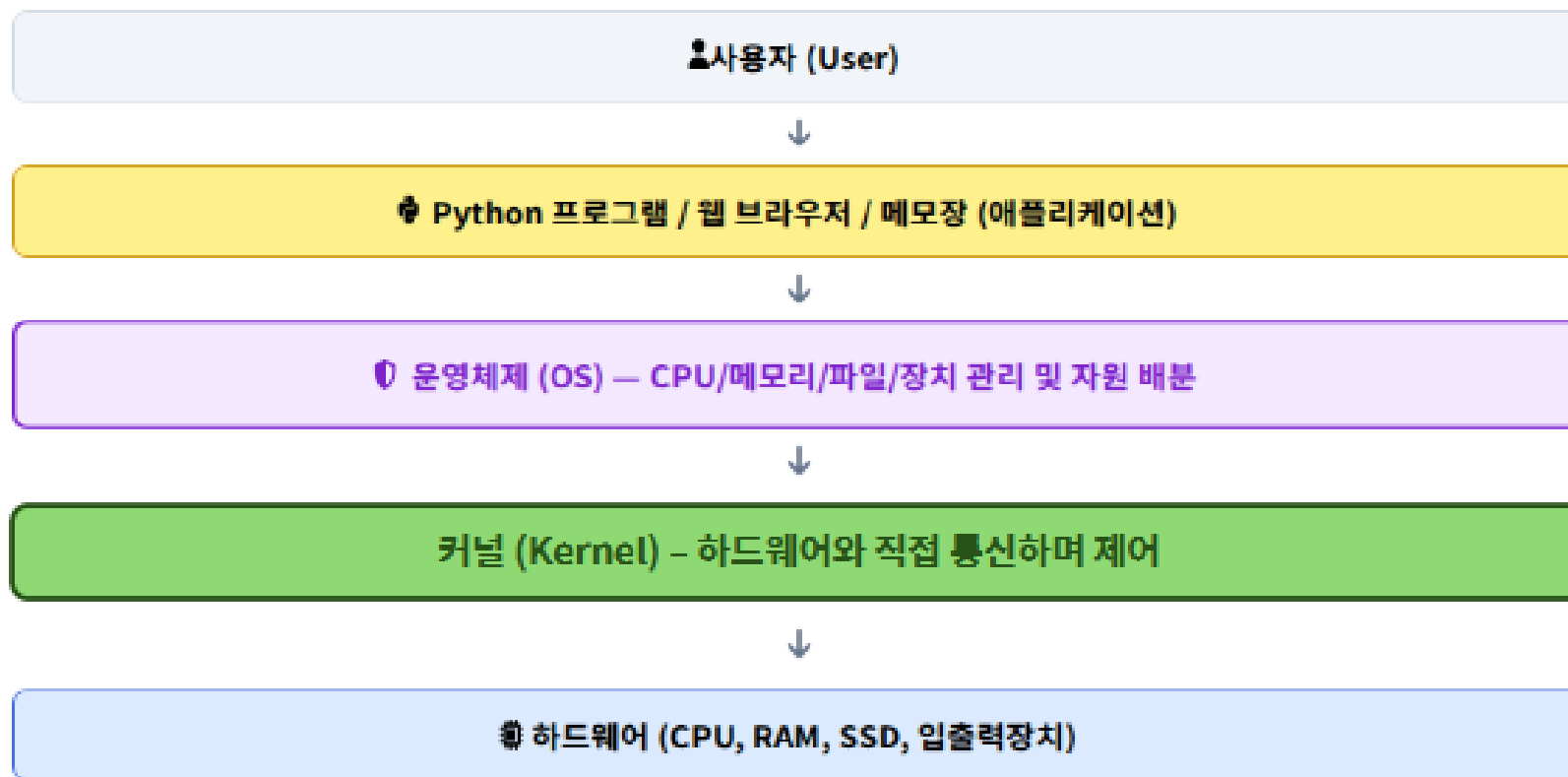
위로 갈수록: 속도 ↑ 가격 ↑ 용량 ↓

아래로 갈수록: 속도 ↓ 가격 ↓ 용량 ↑

CPU는 연산 속도와 저장 공간의 한계를 극복하기 위해 계층 구조 활용

01 컴퓨터 시스템과 운영체제 개요

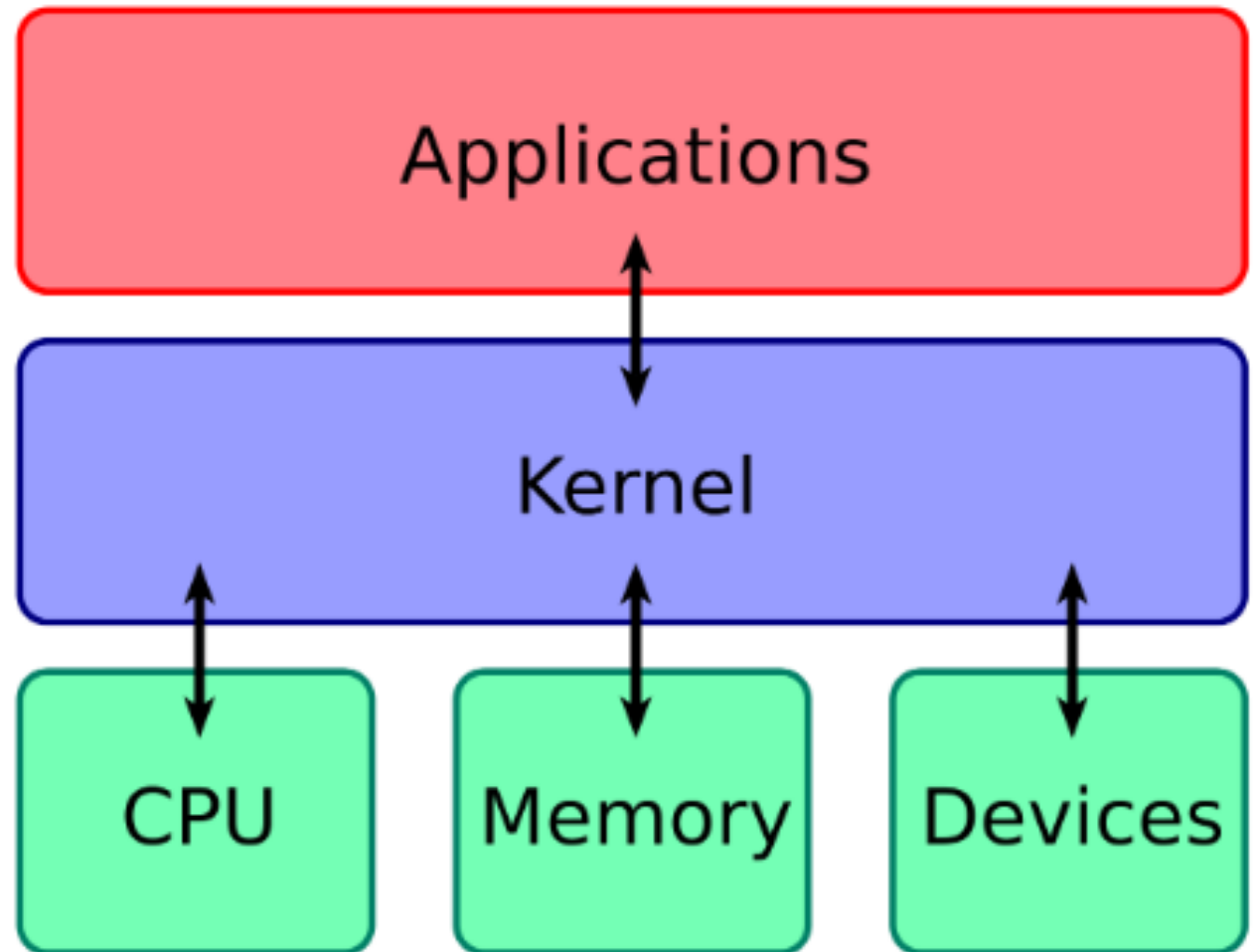
시스템 계층 구조 System Hierarchy



01 컴퓨터 시스템과 운영체제 개요

커널 Kernel

- 운영체제의 핵심인 소프트웨어
- 하드웨어 자원을 자원이 필요한 프로그램에 나눠줌
- 작업 제어, 메모리 제어, 시스템 콜 등 수행
- 하드웨어에 직접적으로 명령
- OS마다 커널 구조가 다 다름

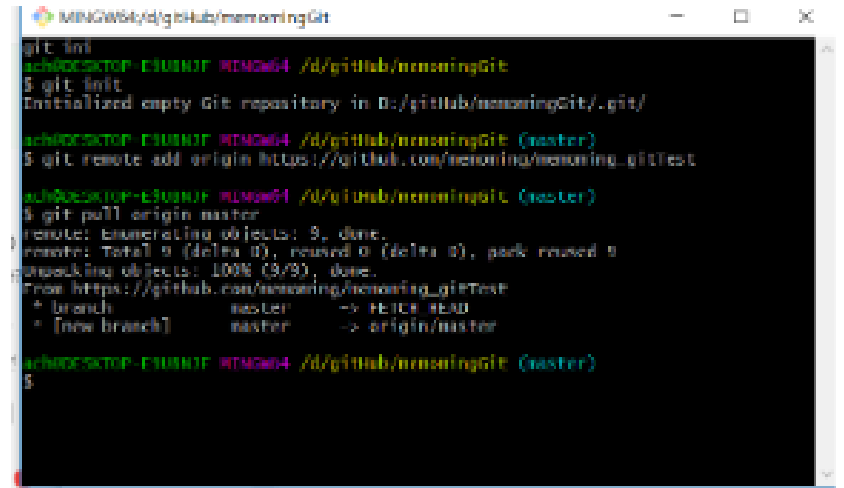


01 컴퓨터 시스템과 운영체제 개요

셸

Shell

- 커널보다 한 단계 바깥의 소프트웨어
- 이용자의 명령을 해석하고 그 결과를 반환
- 사용자가 OS의 핵심(커널)을 둘러싼 껍질을 통해 커널에 접근한다 해서 Shell 이라 부름
- Git Bash, CMD, PowerShell 등이 대표적인 Shell



```
MINGW64/d/github/memoringGit
git init
$ git init
Initialized empty Git repository in D:/github/memoringGit/.git/

$ git remote add origin https://github.com/memoring/memoring_gitTest
$ git pull origin master
remote: Enumerating objects: 9, done.
remote: Total 9 (delta 0), reused 0 (delta 0), pack reused 0
Unpacking objects: 100% (9/9), done.
From https://github.com/memoring/memoring_gitTest
* branch            master       -> FETCH_HEAD
* [new branch]      master       -> origin/master

$
```



```
관리자: 명령 프롬프트
Microsoft Windows [Version 10.0.19045.2364]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator>
```

01 컴퓨터 시스템과 운영체제 개요

시스템 콜 System Call

시스템 콜 (System Call)이란?

"일반 프로그램(User Mode)이 운영체제 커널(Kernel Mode)에게 필요한 하드웨어 작업을 요청하는 공식적인 통로"

손님(파이썬 프로그램)이 주방(커널)에 직접 들어가 요리를 할 수 없으므로 웨이터(시스템 콜)를 통해 주문하는 것과 같음

Python 코드



System Call 요청



OS 커널 처리



하드웨어 장치 제어

CPU 내부 레지스터

Program Counter (PC)

'프로그램 카운터'

다음에 실행해야 할 명령어의 메모리 주소를 기억하고 있는 가장 중요한 레지스터.
매 명령어가 실행될 때마다 자동으로 1씩 증가

Instruction Register (IR)

'명령어 레지스터'

방금 메모리에서 읽어들이는 실제 명령어를 잠시 임시로 보관하여
제어장치가 해석할 수 있도록 대기하는 공간

General Purpose Register

'범용 레지스터'

데이터나 계산 중간 결과, 주소 등을 자유롭게 담아둘 수 있는
다목적 저장 공간

Stack Pointer (SP) 등

'특수 목적 레지스터'

함수 호출이나 임시 메모리 스택 구조를 추적하는 데 사용되는
주소 관리 레지스터 (특수 목적이라고 생각하면 됨)

System BUS

시스템 버스의 종류

CPU, 메모리, 입출력장치 등 장치 사이사이 데이터가 이동하는 물리적인 통로 (여기선 CPU ↔ RAM 만 다름)

Address Bus (주소 버스)

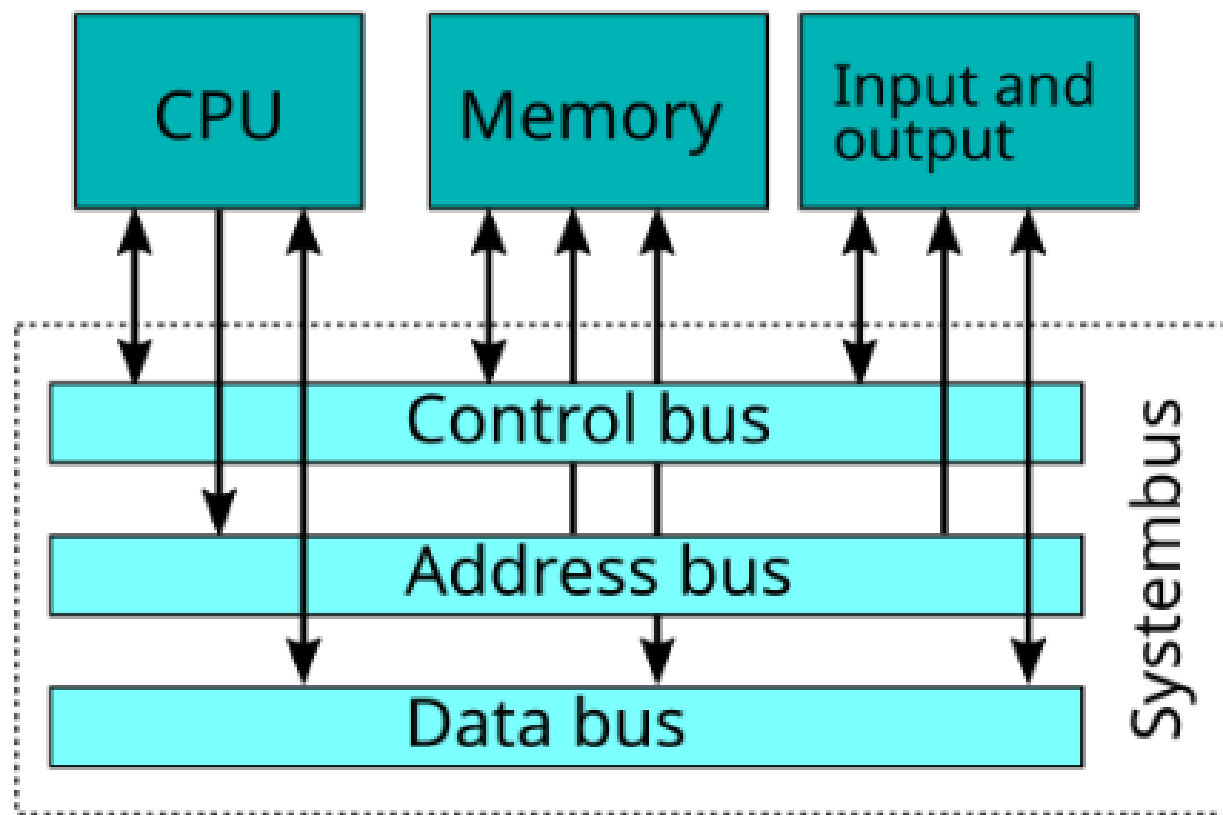
CPU가 지금 접근하려는 메모리의 주소 전달 (단방향, CPU → RAM)

Data Bus (데이터 버스)

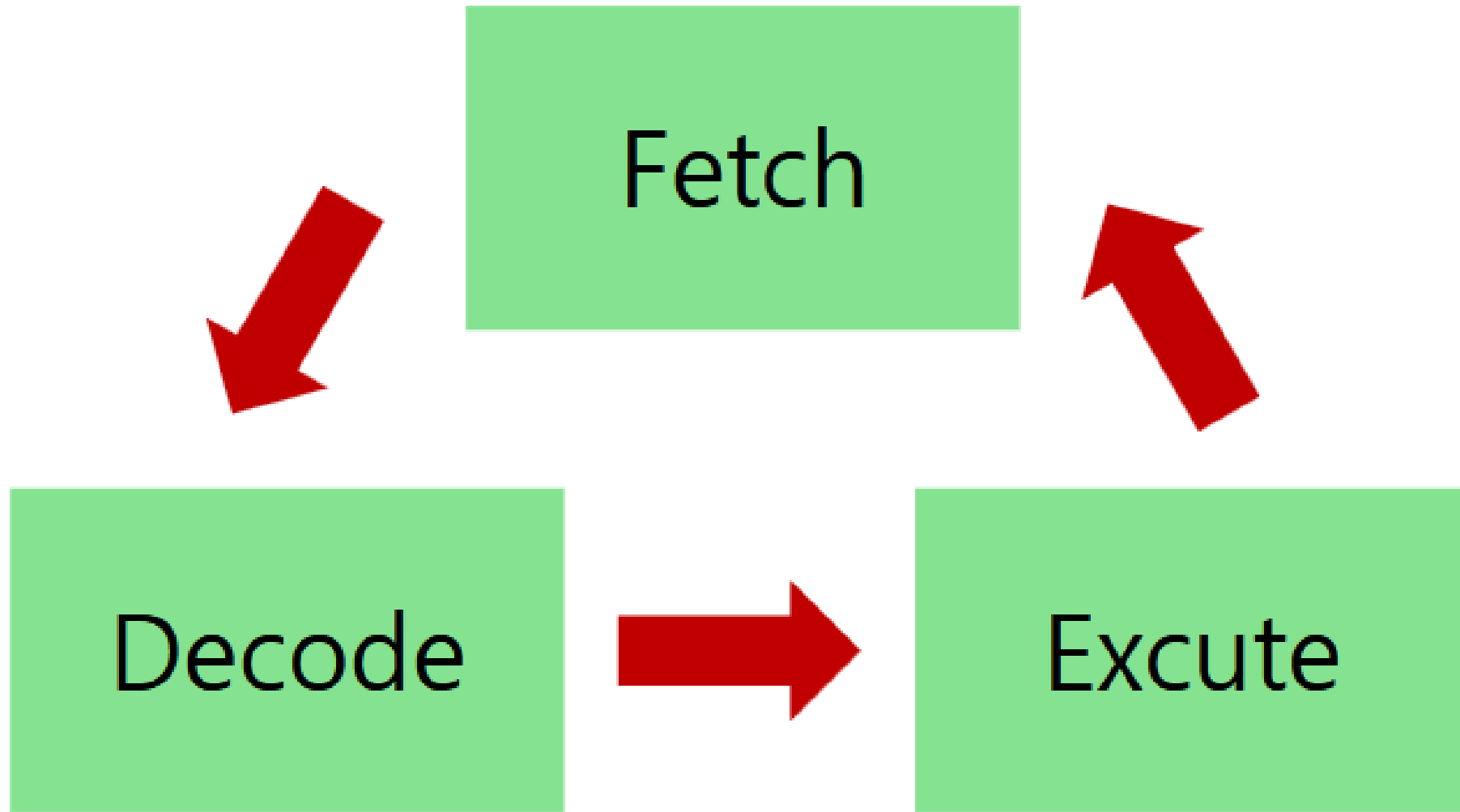
진짜 데이터가 오고가는 통로 (양방향)

Address Bus (주소 버스)

읽기/쓰기 등 제어 신호와 지시 신호 전달 (양방향)



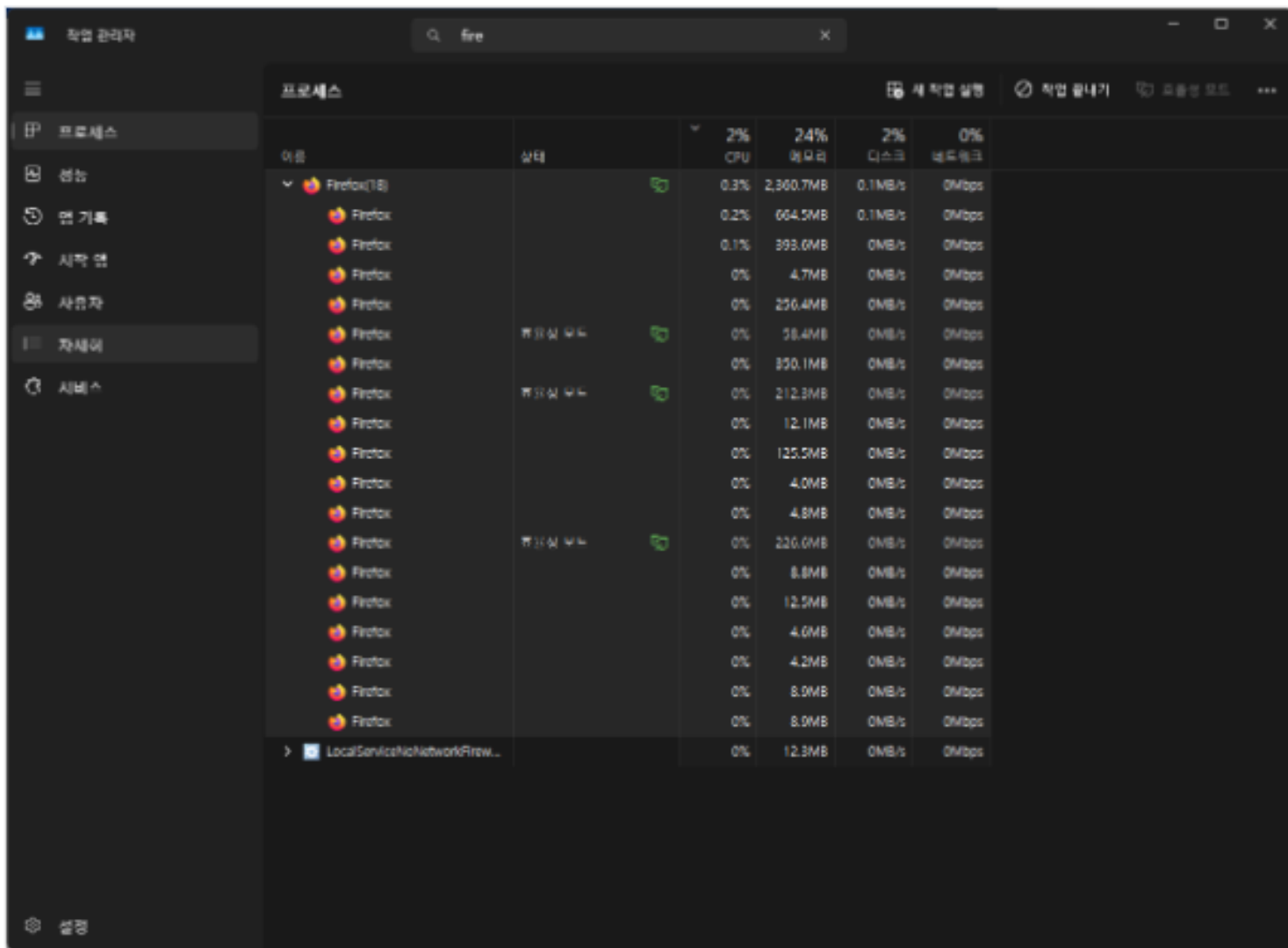
F-D-E Cycle



03 Process & Thread

프로세스

- 윈도우의 경우 작업관리자에서 확인 가능
- SSD에 저장된 Firefox의 코드가 CPU에서 실행되며 18개의 인스턴스(프로세스) 생성
- 각 프로세스마다 하는 일이 다르고, 자원 소모량도 다름
- 리눅스나 맥에서는 어떻게 보는지 몰?루 (알아서 찾아보셈)



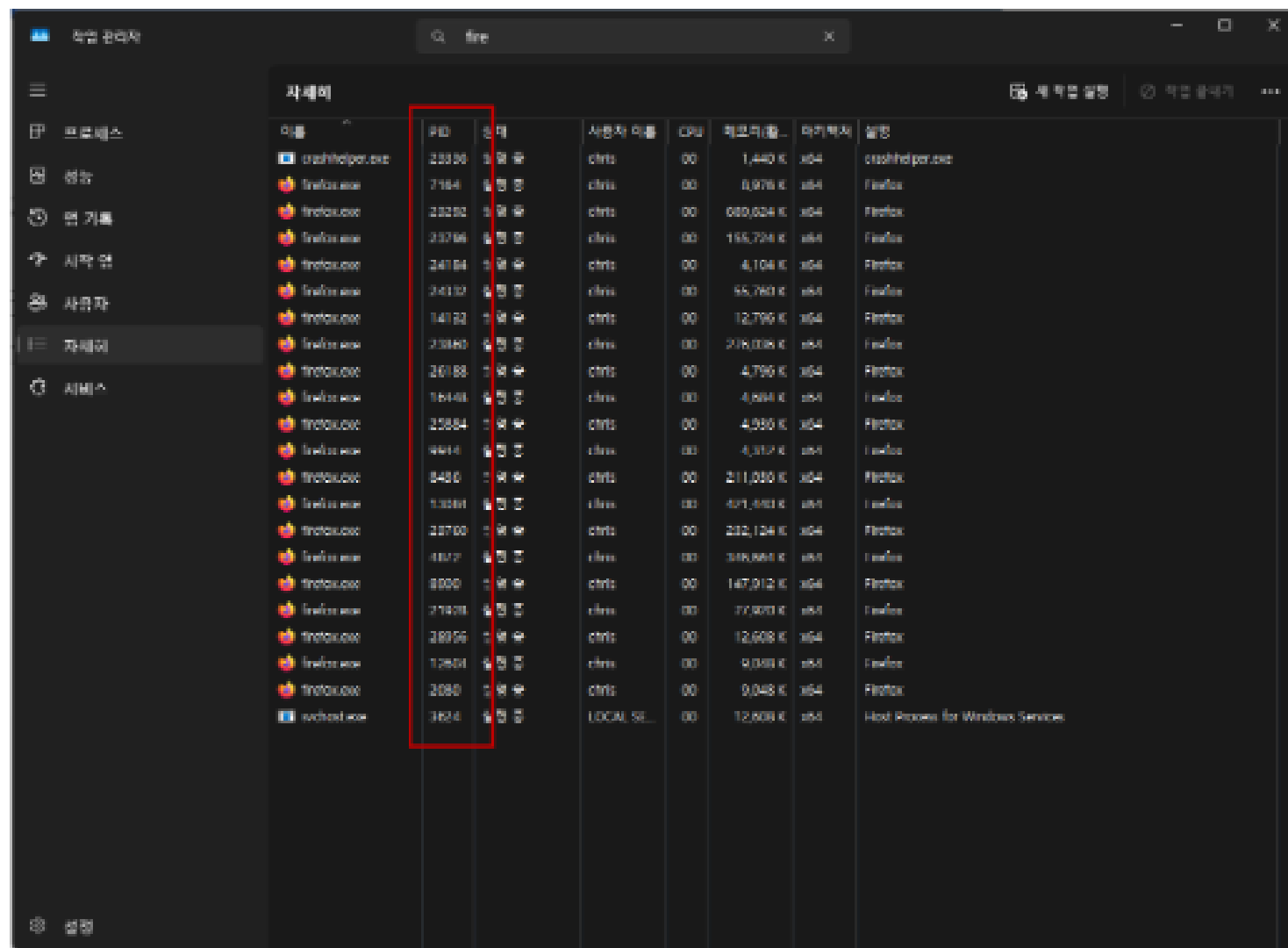
The screenshot shows the Windows Task Manager window with the 'Processes' tab selected. A search bar at the top contains 'fire'. The list of processes shows 18 instances of Firefox, each with its own icon and name. The columns displayed are Name, Status, CPU, Private Memory, Working Set, and Disk I/O. The first instance is 'Firefox(18)' with a CPU usage of 0.3% and 2,360.7MB of private memory. The other instances are all at 0% CPU usage and have varying private memory usage.

이름	상태	CPU	메모리	디스크	네트워크
Firefox(18)		0.3%	2,360.7MB	0.1MB/s	0Mbps
Firefox		0.2%	664.5MB	0.1MB/s	0Mbps
Firefox		0.1%	393.0MB	0MB/s	0Mbps
Firefox		0%	4.7MB	0MB/s	0Mbps
Firefox		0%	256.4MB	0MB/s	0Mbps
Firefox	휴면 상태	0%	58.4MB	0MB/s	0Mbps
Firefox		0%	350.1MB	0MB/s	0Mbps
Firefox	휴면 상태	0%	212.3MB	0MB/s	0Mbps
Firefox		0%	12.1MB	0MB/s	0Mbps
Firefox		0%	125.5MB	0MB/s	0Mbps
Firefox		0%	4.0MB	0MB/s	0Mbps
Firefox		0%	4.8MB	0MB/s	0Mbps
Firefox	휴면 상태	0%	220.0MB	0MB/s	0Mbps
Firefox		0%	8.8MB	0MB/s	0Mbps
Firefox		0%	12.5MB	0MB/s	0Mbps
Firefox		0%	4.0MB	0MB/s	0Mbps
Firefox		0%	4.2MB	0MB/s	0Mbps
Firefox		0%	8.0MB	0MB/s	0Mbps
Firefox		0%	8.0MB	0MB/s	0Mbps
LocalService\NetworkFirew...		0%	12.3MB	0MB/s	0Mbps

03 Process & Thread

PID Process ID

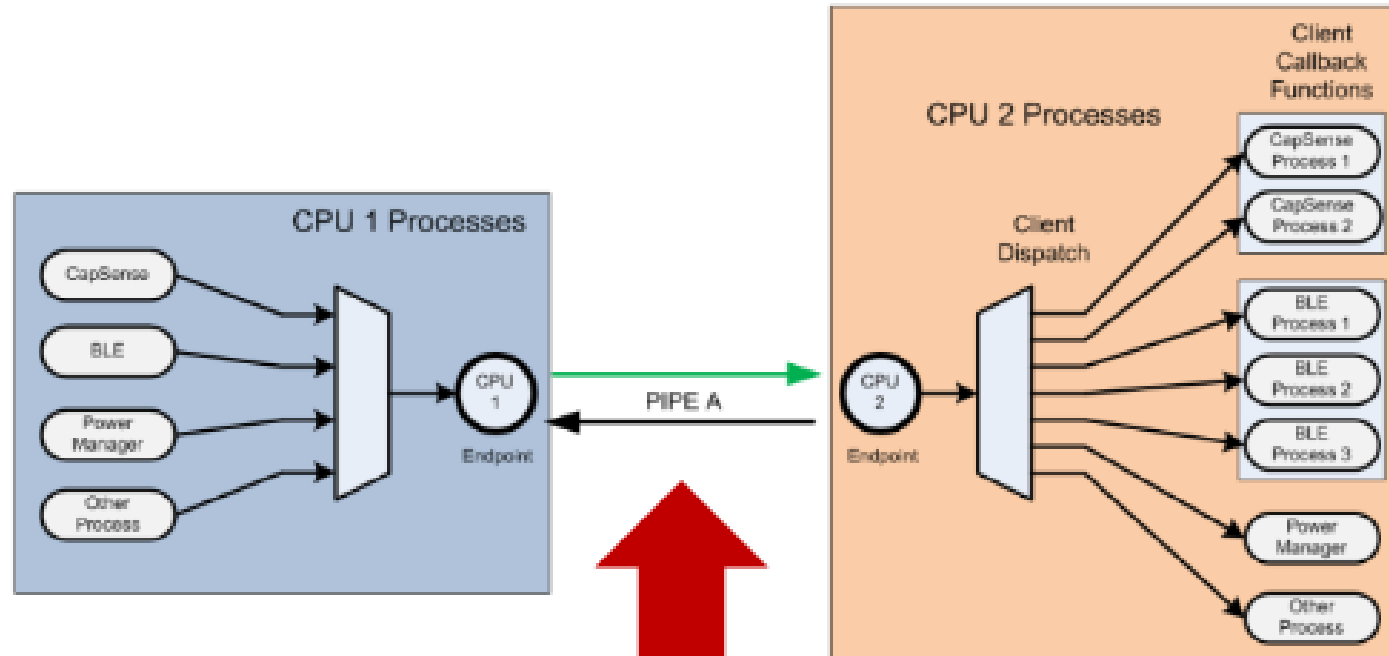
- OS가 여러 프로세스를 구분하기 위해 붙이는 고유한 번호
- 동일한 프로그램을 실행해도 PID는 다름
- OS는 PID로 프로세스를 관리 및 제어함



이름	PID	상태	사용자	이름	CPU	메모리(용량)	작업체	설명
crashhelper.exe	23320	종료됨	chrome		00	1,440 K	x64	crashhelper.exe
firefox.exe	7184	실행 중	chrome		00	0,976 K	x64	Firefox
firefox.exe	23280	종료됨	chrome		00	680,624 K	x64	Firefox
firefox.exe	23284	실행 중	chrome		00	155,724 K	x64	Firefox
firefox.exe	24704	종료됨	chrome		00	4,104 K	x64	Firefox
firefox.exe	24707	실행 중	chrome		00	95,760 K	x64	Firefox
firefox.exe	14132	종료됨	chrome		00	12,796 K	x64	Firefox
firefox.exe	23340	실행 중	chrome		00	206,036 K	x64	Firefox
firefox.exe	26188	종료됨	chrome		00	4,796 K	x64	Firefox
firefox.exe	16408	실행 중	chrome		00	4,804 K	x64	Firefox
firefox.exe	25884	종료됨	chrome		00	4,980 K	x64	Firefox
firefox.exe	9414	실행 중	chrome		00	4,312 K	x64	Firefox
firefox.exe	8420	종료됨	chrome		00	211,030 K	x64	Firefox
firefox.exe	13380	실행 중	chrome		00	471,560 K	x64	Firefox
firefox.exe	23700	종료됨	chrome		00	210,124 K	x64	Firefox
firefox.exe	4872	실행 중	chrome		00	306,864 K	x64	Firefox
firefox.exe	8000	종료됨	chrome		00	147,012 K	x64	Firefox
firefox.exe	27408	실행 중	chrome		00	77,000 K	x64	Firefox
firefox.exe	28956	종료됨	chrome		00	12,608 K	x64	Firefox
firefox.exe	12804	실행 중	chrome		00	9,080 K	x64	Firefox
firefox.exe	2680	종료됨	chrome		00	9,048 K	x64	Firefox
svchost.exe	3624	실행 중	LOCAL SE...		00	12,808 K	x64	Host Process for Windows Services

03 Process & Thread

프로세스 간 데이터 통신: IPC



프로세스 A

프로세스 B

프로세스 간 데이터 통신

03 Process & Thread

스레드 Thread

- 사용자가 파이어폭스를 한 번 실행했을 때 OS가 몰래 프로세스를 17개 생성함
- PID 28650 프로세스는 스레드를 121개, PID 276 프로세스는 스레드를 31개 가진다는 뜻
- 하나의 프로세스는 여러 개의 스레드를 가질 수 있음
- 프로세스와 유사하게 각 스레드마다 ID(TID)를 가짐

```
Windows PowerShell
[spyder](base) PS C:\WINDOWS\system32> Get-Process firefox | Select-Object Id, ProcessName, Threads.Count

Id ProcessName Threads
--
276 firefox      31
836 firefox      30
1636 firefox     27
2048 firefox     33
7728 firefox     27
9084 firefox      7
9968 firefox     40
11000 firefox    33
15852 firefox    10
19072 firefox    87
21736 firefox      7
27280 firefox    31
27792 firefox    27
28560 firefox   121
28764 firefox    27
32584 firefox      8
33572 firefox      8

[spyder](base) PS C:\WINDOWS\system32>
```

실제로 파워셸에서 파이어폭스의 스레드 목록을 출력해봄

03 Process & Thread

스레드가 공유하는 것 · 공유하지 않는 것

공유하는 것 (Shared)

- Code / Text 영역
- Data 영역 (전역 변수 등)
- 프로세스 내부 메모리
- 열린 파일 및 OS 자원

공유하지 않는 것 (Independent)

- 각자 호출한 함수와 지역 변수
- Register 상태 (PC 포함, 각자의 실행 위치)
- Thread ID

Context Switching

프로세스의 저장 공간

정의: CPU가 실행 중인 프로세스(또는 스레드)를 바꾸기 위해 현재 상태를 저장하고 다음 프로세스의 상태를 복원하는 과정

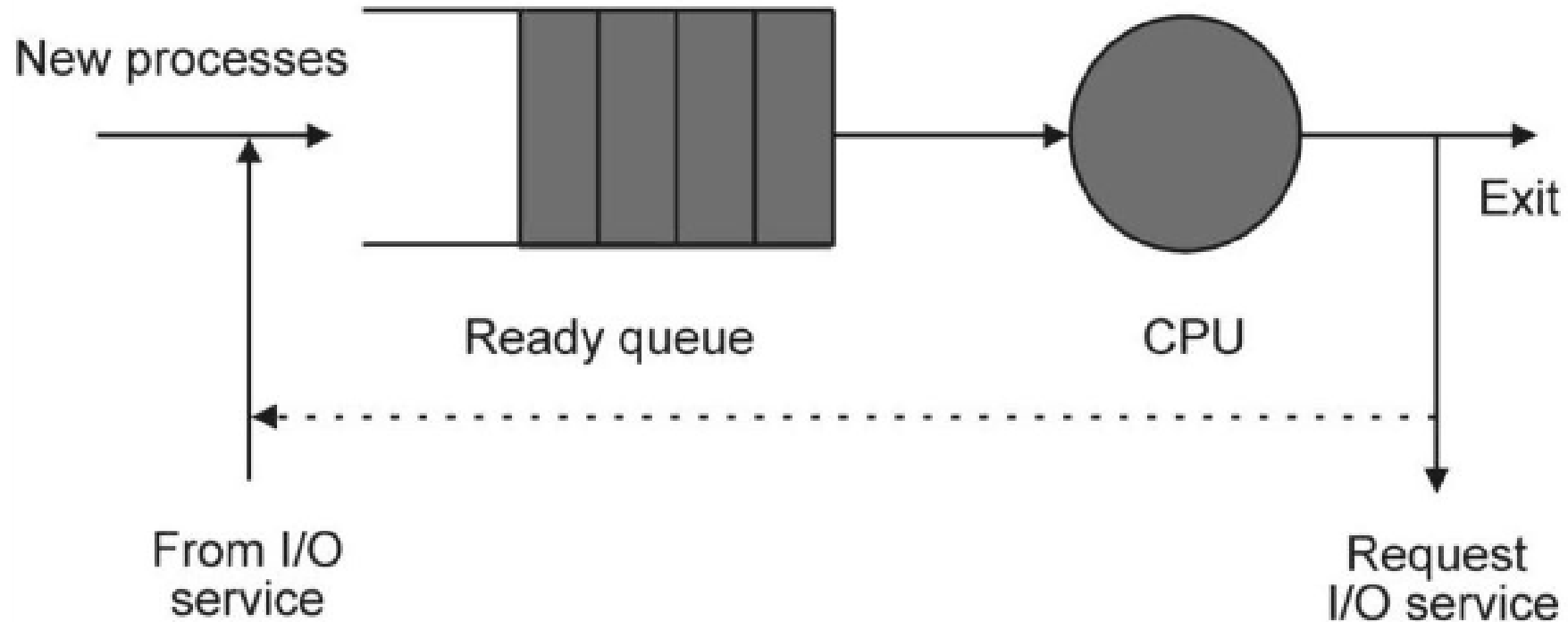


과정: Process A 실행 중단 → A의 Context(레지스터, PC 등)를 PCB에 백업 → 운영체제 스케줄러 작동 → Process B의 PCB에서 Context 복원 → Process B 실행

```
Thread A 실행
  ↓
[A의 CPU 실행 상태 저장]
  ↓
[B의 CPU 실행 상태 복원]
  ↓
Thread B 실행
```

04 CPU Scheduling

Ready Queue



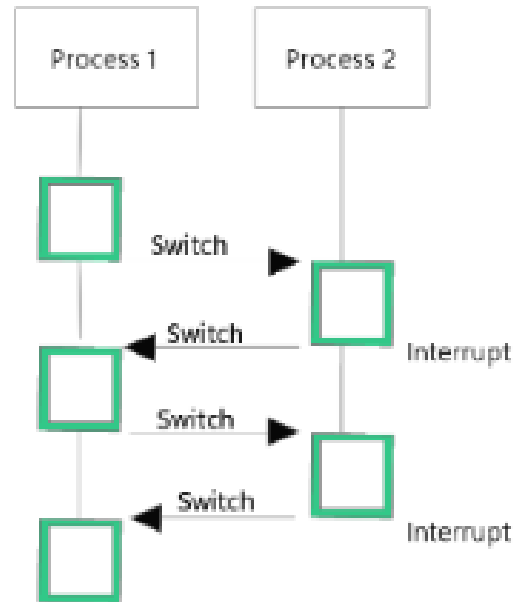
실행될 프로세스를 큐에 모아놓음
그런 다음 선착순으로 실행
그 큐를 Ready Queue라고 함

04 CPU Scheduling

Preemptive & Non-Preemptive

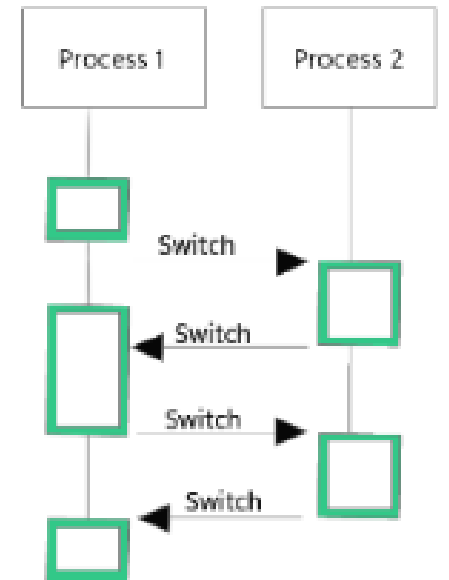
- CPU 자원을 받은 프로세스는 종료되거나 대기 상태에 들어가면 할당받은 CPU를 놓아줌
- 이렇게 프로세스가 CPU를 놓아줬을때만 다른 프로세스로 할당시키는걸 비선점, Non-Preemptive라고 함
- 반면 OS가 실행 중인 프로세스의 CPU를 강제로 뺏아서 다른 프로세스에 할당하는 경우도 있음
- 이를 선점, Preemptive라고 함

Preemptive Scheduling



vs

Non-Preemptive Scheduling

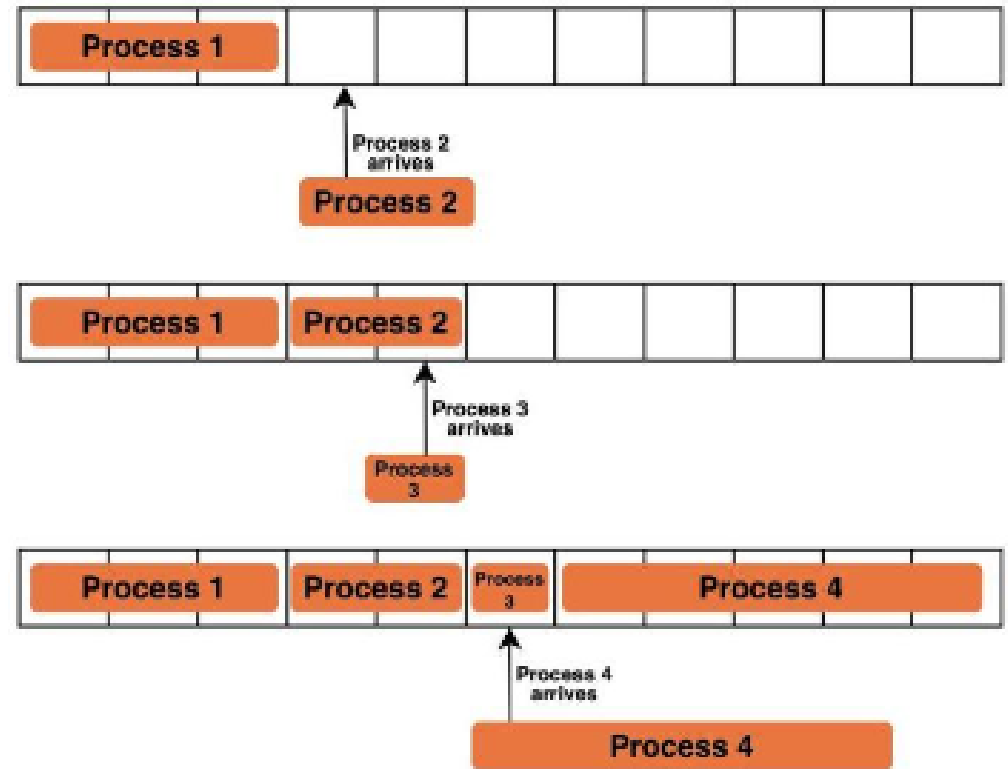


04 CPU Scheduling

FCFS: First Come, First Served

- 선착순 방식
- Non-Preemptive 방식
- 구현이 매우 단순함 (단순 Queue)
- 한번 CPU를 잡으면 프로세스가 놓기 전까지 뺏기지 않음
- 모든 프로세스가 언젠가는 실행됨
- 긴 실행 시간의 프로세스 뒤에 있는 프로세스는 한참 기다려야 하는 단점이 있음

First Come First Serve



04 CPU Scheduling

SJF: Shortest Job First

[예제] P1(6ms), P2(8ms), P3(7ms), P4(3ms) 모두 0ms 도착



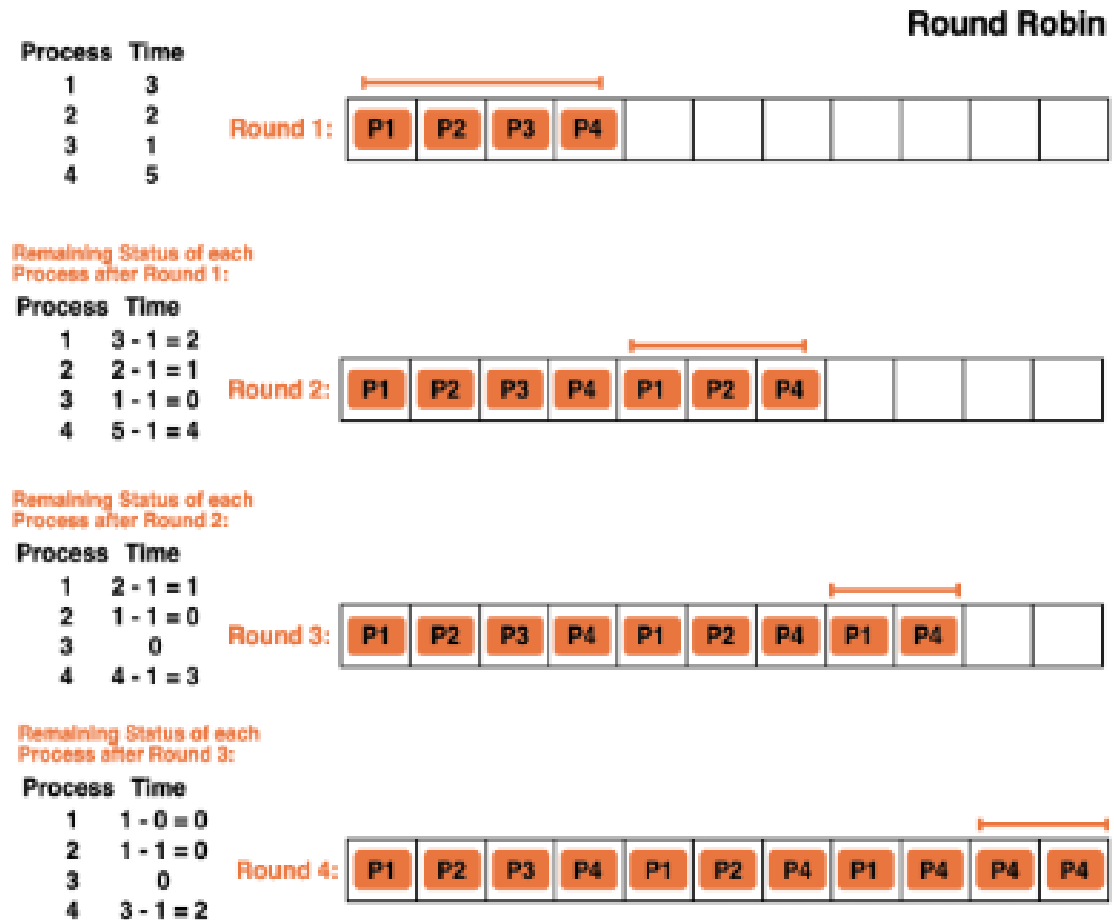
SJF 대기 시간: P4=0ms, P1=3ms, P3=9ms, P2=16ms → 평균 대기 시간: 7.0 ms
(참고: FCFS 적용 시 P1→P2→P3→P4 순서로 평균 대기 시간 10.25 ms 발생)

실행 시간 짧은 놈부터 실행
(실행시간 똑같으면 먼저 온 놈부터)
아도 Non-Preemptive 방식임

04 CPU Scheduling

Round Robin

- 이번엔 CPU를 쓸 수 있는 시간을 제한해봄
- 그걸 Time Quantum이라 함
- 프로세스가 CPU를 사용하며 실행되다가 Time Quantum이 지나면 CPU 할당을 빼앗기고 Ready Queue 맨 뒤로 강제 이동됨
- Time Quantum은 보통 수 ms ~ 수십 ms 정도 됨
- Time Quantum이 너무 짧으면 Context Switching 비용이 폭발하고, 너무 길면 FCFS랑 비슷해져서 성능이 좀 떨어짐
(트레이드오프를 고려해서 설계해야됨)
- 현대에는 대부분 이 방식 사용함



05 프로세스 동기화

동시성 문제



공동계좌 잔액: 10만원



두 명이 동시에 출금하는 상황

동시성 문제



각각 10,000원씩 출금

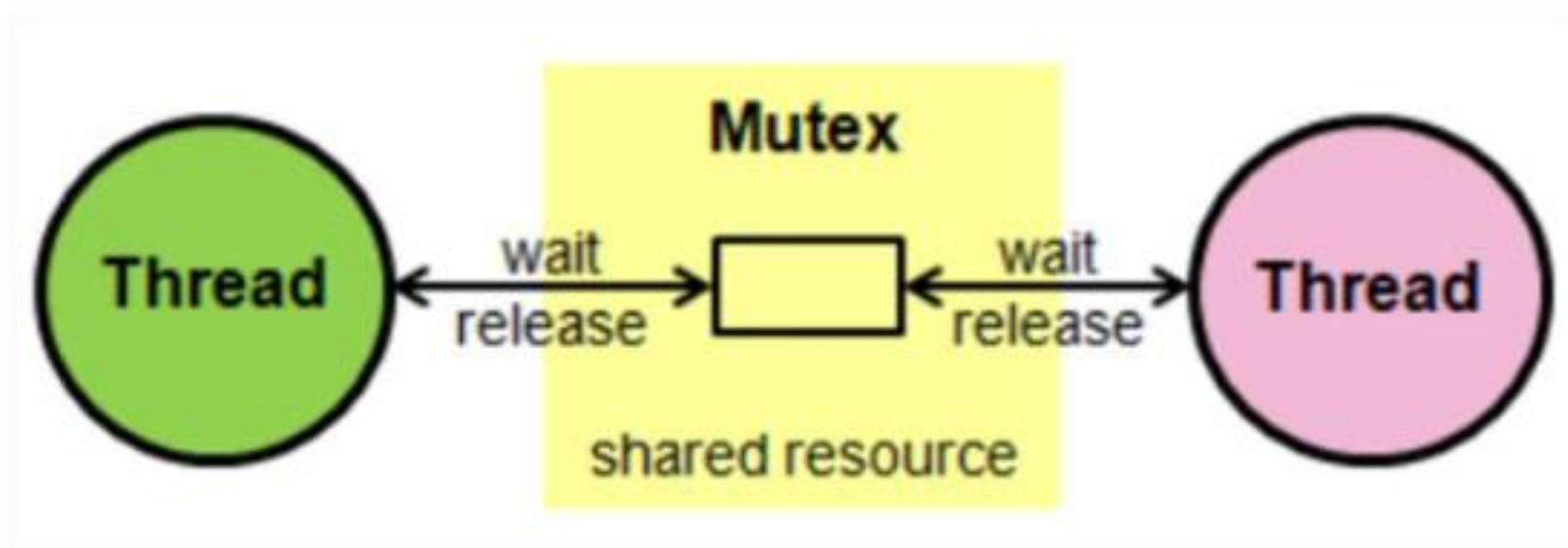


공동계좌 잔액: 10만원
→ 동시에 출금한 순간 10,000 출금 실행
→ 계좌에 남은 금액은 총 90,000원



뭔가 이상한듯

Mutex: MUTual EXclusion



스레드는 Critical Section(화장실)에 진입하기 전 문을 잠금 (Lock이라 부름)
만약 이미 Lock 된 Critical Section이라면 락 풀릴때까지 대기 (Block이라 부름)
볼일 끝나면 잠금 해제 (Release라 부름)

05 프로세스 동기화

Semaphore

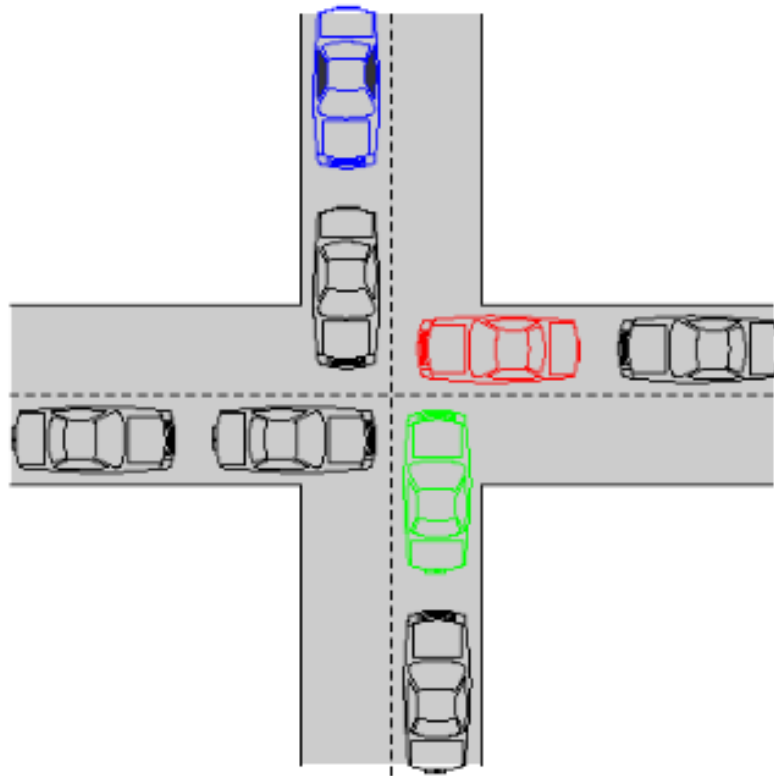


Block 상태



빈 방에 무조건 1명만 들어가야 했던 Mutex 와 달리
Semaphore의 빈 방에는 N명까지 들어갈 수 있음
주차 공간이 2개인 주차장에서 3번째 차는
공간 날때까지 기다리는거 생각하면 바로 이해될걸

Deadlock

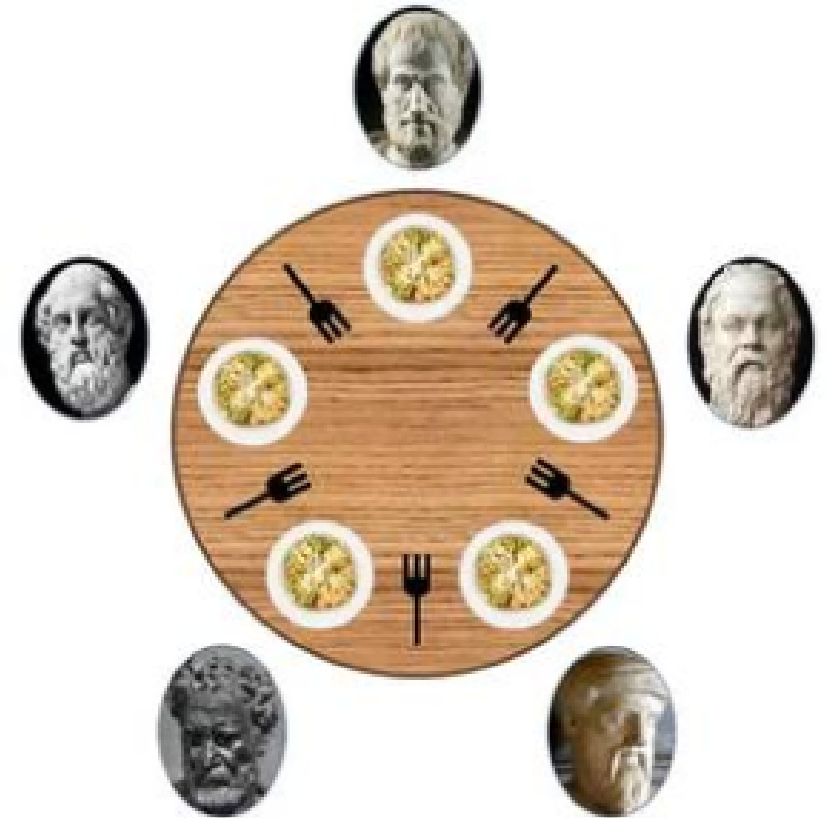


이렇게 동작이 꼬여서 아무도 못 움직이는 상황을
Deadlock, 교착 상태라 부름

Deadlock 의 고전적인 비유

- Deadlock을 설명할 때 가장 널리 쓰이는 비유로 식사하는 철학자 문제라는게 있음
1965년에 다익스트라 라는 아저씨가 만듦
(참고사항으로, 자료구조 12주차에 다익스트라 알고리즘 할건데 그것도 이 아저씨가 만듦)
- 이 문제에서 가정하는 상황은 아래와 같음
 - 다섯 명의 철학자가 원탁에 앉아서 식사를 한다
 - 각 철학자 사이에는 포크가 하나씩 있고, 앞에는 스파게티가 담긴 접시가 있다
 - 메뉴가 스파게티인 관계로 무조건 양손에 포크를 하나씩 쥐고 먹어야만 한다
 - 각 철학자는 식사 또는 생각만 할 수 있고 말은 할 수 없다

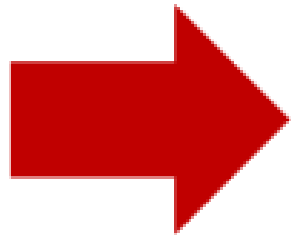
사실 이거 봐도 이해 잘 안될텐데
너무 고전적이고 유명한 비유라서 넣어봄



Dining philosophers problem

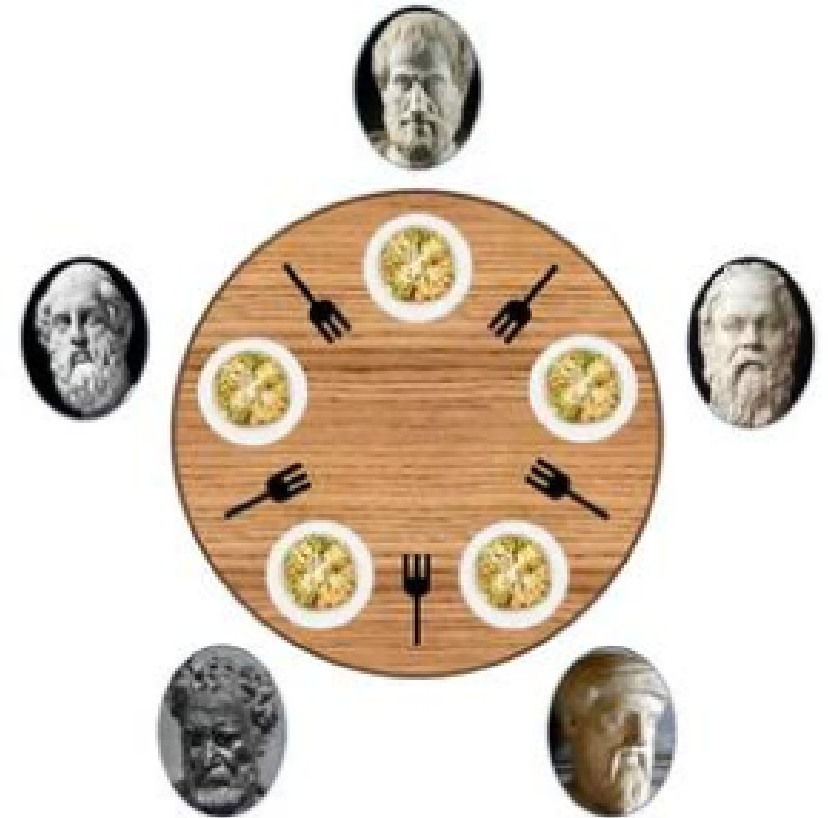
Deadlock 의 고전적인 비유

- 따라서, 철학자가 밥을 먹으려면 왼쪽과 오른쪽의 철학자가 모두 식사를 하지 않는 중이어야만 한다
- 또한, 식사를 마치면 포크를 다른 철학자가 쓸 수 있도록 내려놓는다
- 각 철학자는 서로가 배가 고픈지 아니면 단지 생각하는 중인지 알 수 없다



이 때 어떤 철학자도 굶지 않고
식사할 수 있는 방법은?

사실 이거 봐도 이해 잘 안될텐데
너무 고전적이고 유명한 비유라서 넣어봄



Dining philosophers problem

Deadlock 의 고전적인 비유

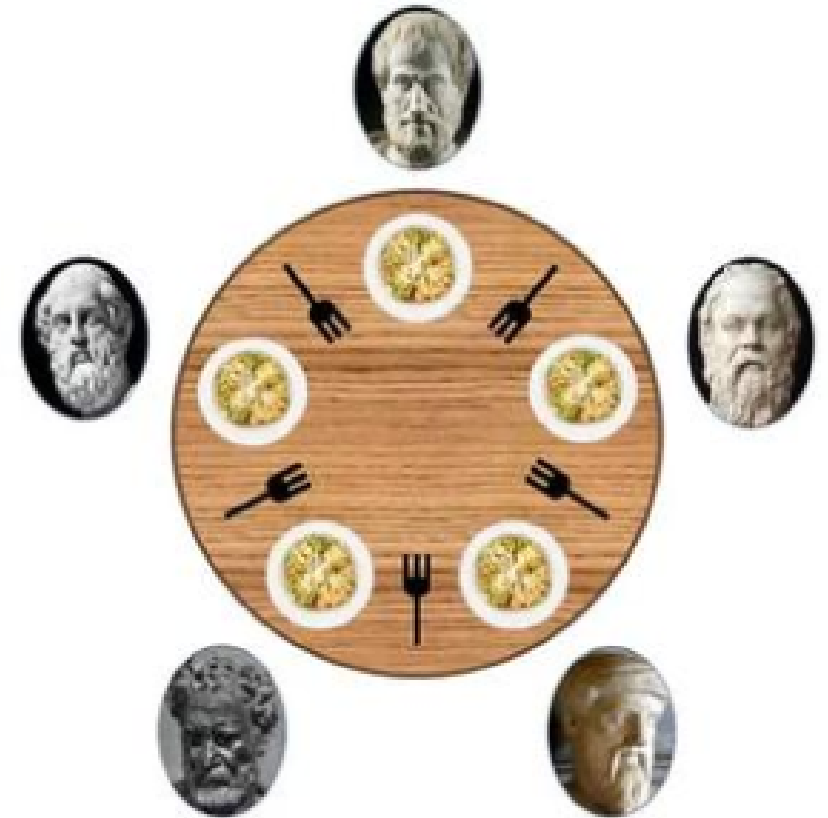
- 일단 각 철학자는 왼쪽 포크부터 집어든 다음, 오른쪽 포크가 사용 가능해질 때까지 대기한다고 하자 (사용 가능하면 집어 들고 식사함)

**모든 철학자가 왼쪽 포크만
집어 들고 식사를 못하게 됨**

- 이번에는 각 철학자가 양 쪽의 포크를 한번에 집어 든다 하자
이렇게 하면 일단 식사는 가능할거같음

**근데 일부 철학자는 밥을 못먹고
무한정 굶게 됨 (Starvation)**

사실 이거 봐도 이해 잘 안될텐데
너무 고전적이고 유명한 비유라서 넣어봄



Dining philosophers problem

Coffman Condition

1. Mutual Exclusion (상호 배제)

자원은 한 번에 한 프로세스만 사용 가능

2. Hold and Wait (점유와 대기)

자원을 전 상태에서 다른 자원을 추가로 요구하며 대기

3. No Preemption (비선점)

다른 프로세스의 자원을 강제로 빼앗을 수 없음

4. Circular Wait (환형 대기)

프로세스간 자원 대기 관계가 원형 고리를 형성

위 4개의 조건이 동시에 만족되면
Deadlock 발생 가능함

(하나라도 깨지면 방지 가능)

Banker's Algorithm

잔고: 30만원



현재 잔고로 A에게 빌려준다면?

1. A가 일을 끝내고 70만원을 갚음
2. 그 70만원으로 B에게 30만원을 빌려줌

→ A에게 30만원 빌려줘도 될듯

Safe State



A: 30만원 필요



B: 30만원 필요

06 Deadlock

Banker's Algorithm

잔고: 10만원



현재 잔고로 A에게 빌려준다면?

내가 가진 돈이 부족한데?

A가 돈을 갚을 수 없는데?

A랑 B는 계속 대기할 수밖에 없는데?

Unsafe State

Deadlock



A: 30만원 필요



B: 30만원 필요

Ostrich Algorithm

- 이걸 알고리즘이라고 불러야 되나 싶긴 한데 암튼
- Deadlock이 발생하는 빈도가 매우 드물수 있음
- 그럼 Deadlock이 실제로 발생했을 때
그냥 재부팅 또는 해당 프로그램만 꺼버림
- 그게 예방, 회피, 감지를 계속 수행하는
비용보다 훨씬 경제적이었음
- 현대의 Windows, Linux, macOS가
이 방법을 채택함



02

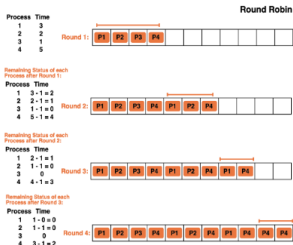
메모리

Memory

프로세스의 메모리 공유

Round Robin

- 이벤트 CPU를 쓸 수 있는 시간을 제한해봄
- 그걸 Time Quantum이라 함
- 프로세스가 CPU를 사용하며 실행되다가 Time Quantum이 지나면 CPU 할당을 빼앗기고 Ready Queue 맨 뒤로 강제 이동됨
- Time Quantum은 보통 수 ms ~ 수십 ms 정도 됨
- Time Quantum이 너무 짧으면 Context Switching 비용이 폭발하고, 너무 길면 FCFS랑 비슷해져서 성능이 좀 떨어짐 (트레이드오프를 고려해서 설계해야됨)
- 현대에는 대부분 이 방식 사용함



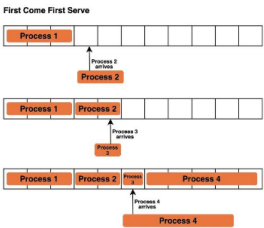
SJF: Shortest Job First



실행 시간 짧은 놈부터 실행
(실행시간 똑같으면 먼저 온 놈부터)
애도 Non-Preemptive 방식임

FCFS: First Come, First Served

- 선착순 방식
- Non-Preemptive 방식
- 구현이 매우 단순함 (단순 Queue)
- 한번 CPU를 잡으면 프로세스가 놓기 전까지 뺏기지 않음
- 모든 프로세스가 언젠가는 실행됨
- 긴 실행 시간의 프로세스 뒤에 있는 프로세스는 한참 기다려야 하는 단점이 있음



CPU 나눠쓰기

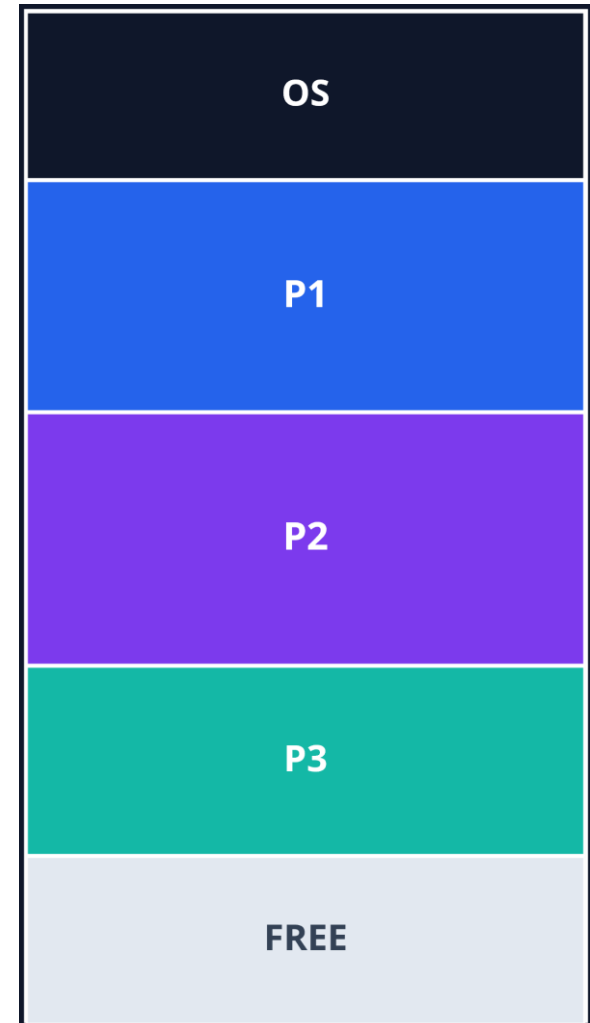
CPU 스케줄링

메모리 나눠쓰기

???

프로세스의 메모리 공유

- 프로세스가 실행되는 동안 서로 메모리 간섭이 생기지 않게 격리 및 보호가 필요함
- 각 프로세스가 자신의 메모리 공간을 독립적으로 사용해야 함

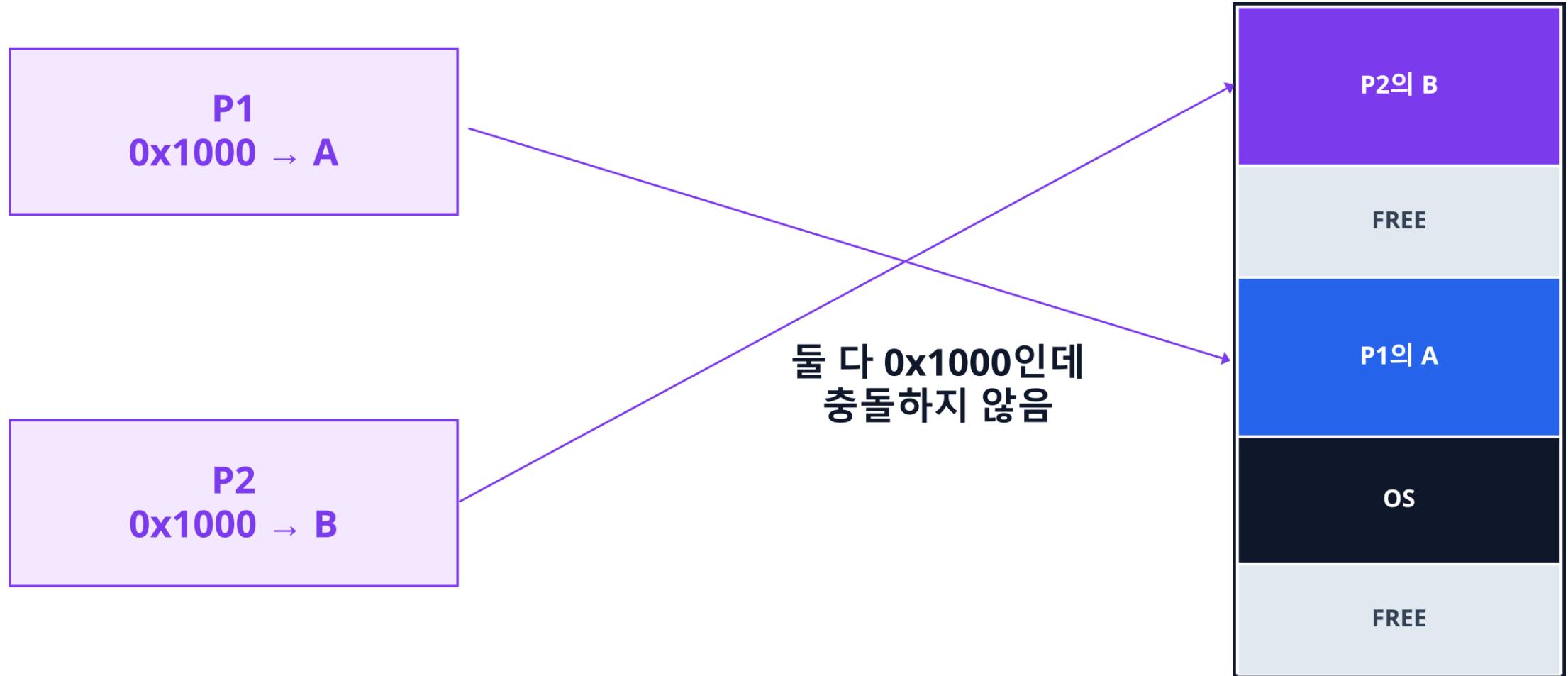


프로세스의 메모리 공유



프로세스가 보는 주소 공간과
실제 RAM의 주소는 다름

논리 주소를 쓰는 이유



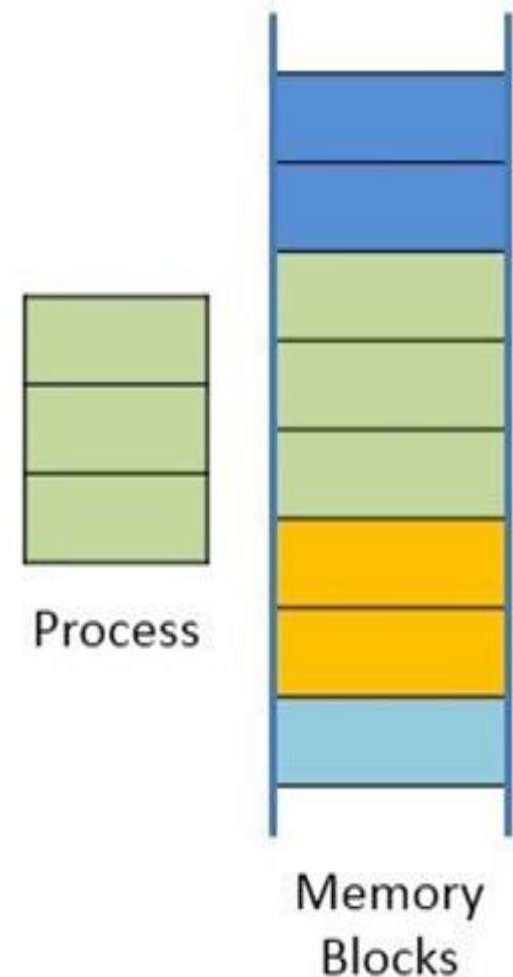
Process Address Space

- 프로세스에게 제공되는 메모리의 구조
- 실행할 명령어(Code), 전역/정적 변수(Data), 동적 변수 및 데이터(Heap), 함수 호출 구역(Stack)으로 구성됨
- 여기에서의 메모리 주소는 논리 주소 기준임
- **여기의 Heap은 알고리즘의 Heap과 완전히 다른 개념**



프로세스에게 메모리 할당하기: Contiguous Allocation

- Continuous: 연속된 (시간 관점)
Contiguous: 인접한 (공간 관점)
- 프로세스 하나를 메모리 주소가 인접하게 배치
- 가장 단순하고, 인접한 영역을 점유하기 때문에 관리하기 쉬움
- 프로세스가 생성되고 종료되는게 반복된 후
메모리 중간중간 쓰이지 않는 부분이 있을 수 있음
(Fragmentation 문제)



03

페이징

Paging

Fixed Partitioning



파티션을 100MB로 나누면 70메가짜리 프로세스는
30메가를 낭비하게 됨

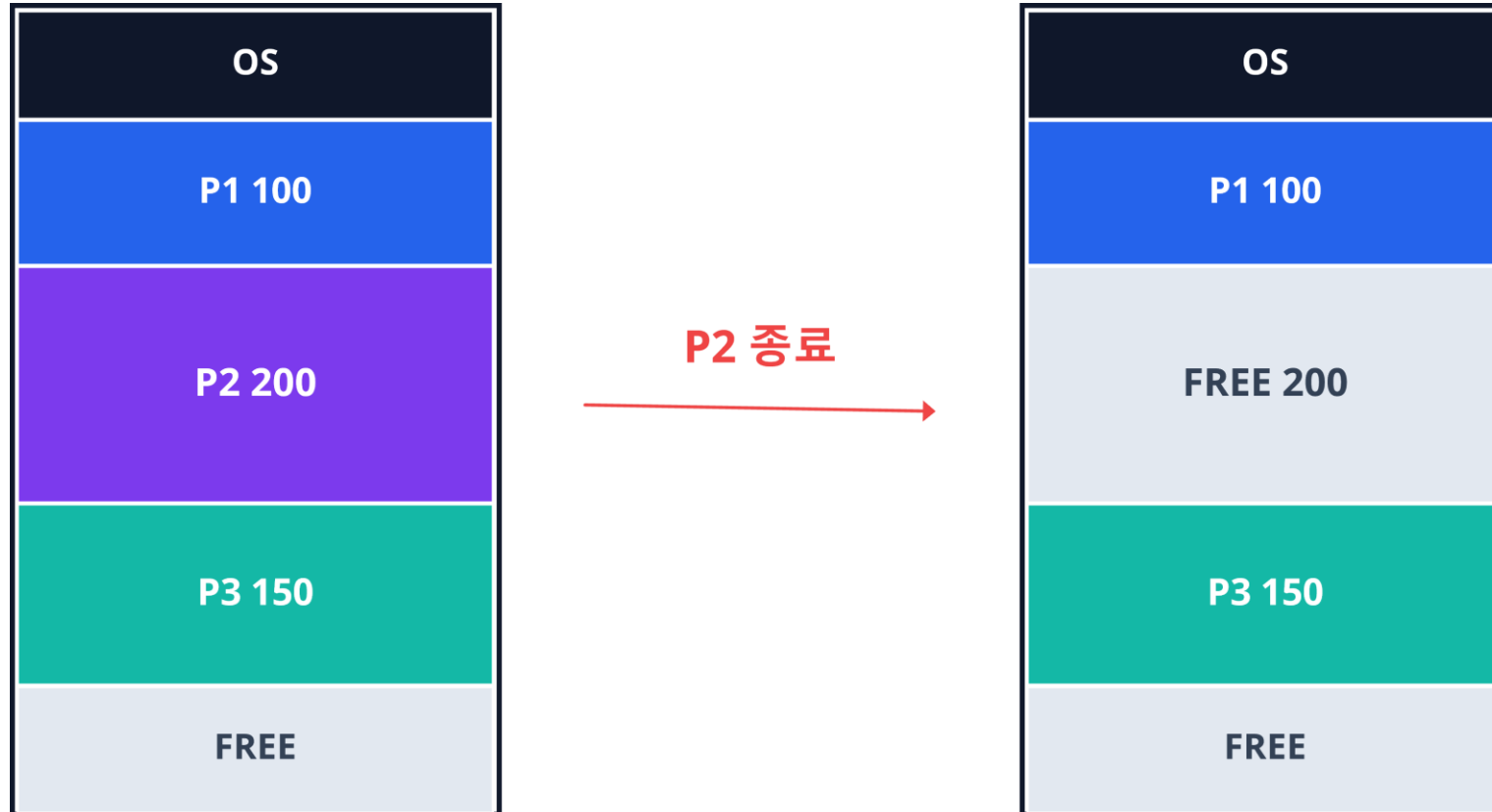
Internal Fragmentation



INTERNAL = 할당된 영역 “안쪽”의 낭비

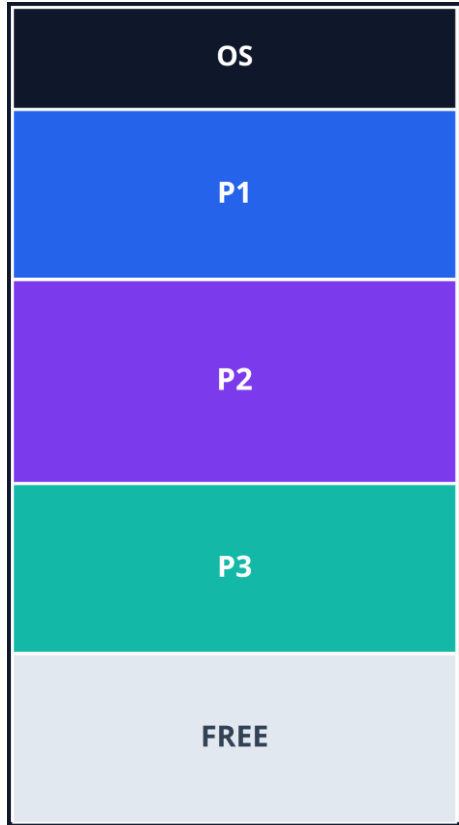
해결법: 프로세스의 크기에 딱 맞춰 파티션 생성

Variable Partitioning

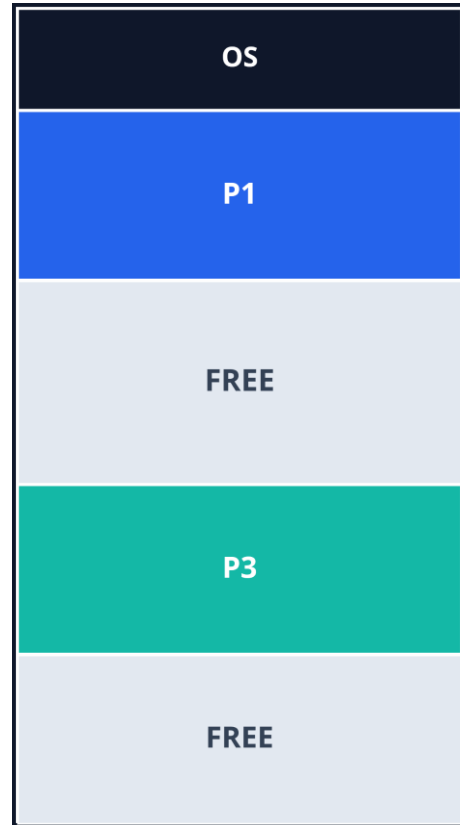


그래서 프로세스 크기에 딱 맞게 파티션 크기를 유동적으로 생성함

Variable Partitioning



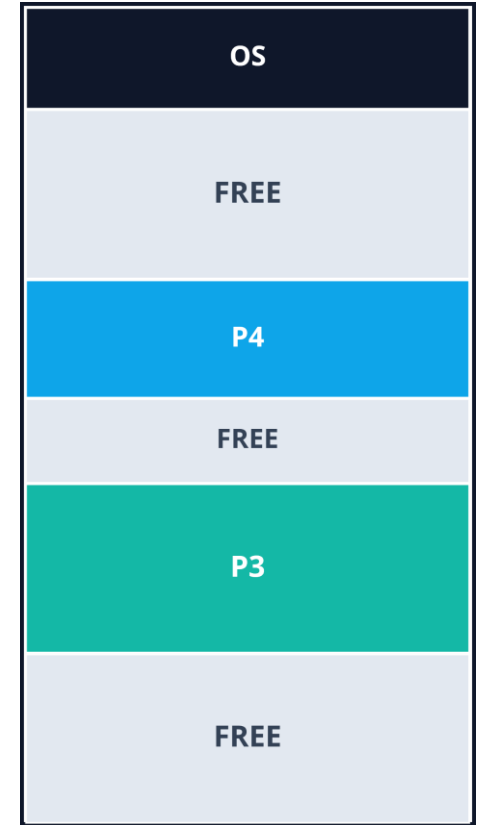
초기 상태



P2 종료



P4 실행



P1 종료

External Fragmentation



NEW P5 · 100MB

Allocation Failed

전체 빈 공간은 충분하나
100MB짜리 인접한 공간이 없음
→ 메모리 할당 실패

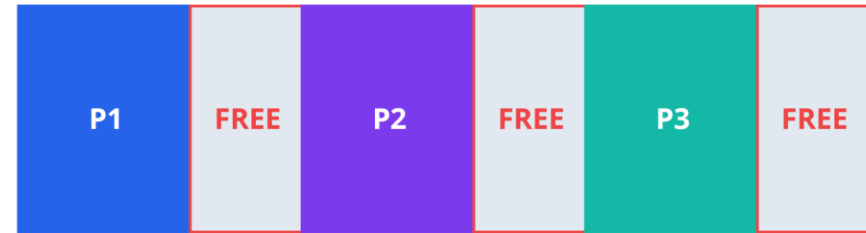
Internal / External Fragmentation

INTERNAL · Block 안



할당된 Block 내부의 낭비

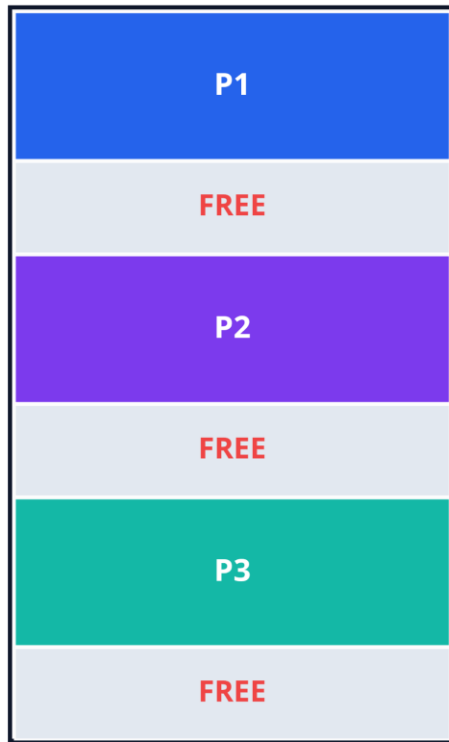
EXTERNAL · Block 사이



Free Space가 여러 곳에 분산

Compaction

BEFORE



MOVE

AFTER



큰 연속 공간 확보

하지만 Memory 내용을 이동하는 비용이 발생

**Process 전체가 반드시 하나의 연속된
Physical Memory 공간에 있어야 할까?**

PROCESS = 큰 Block



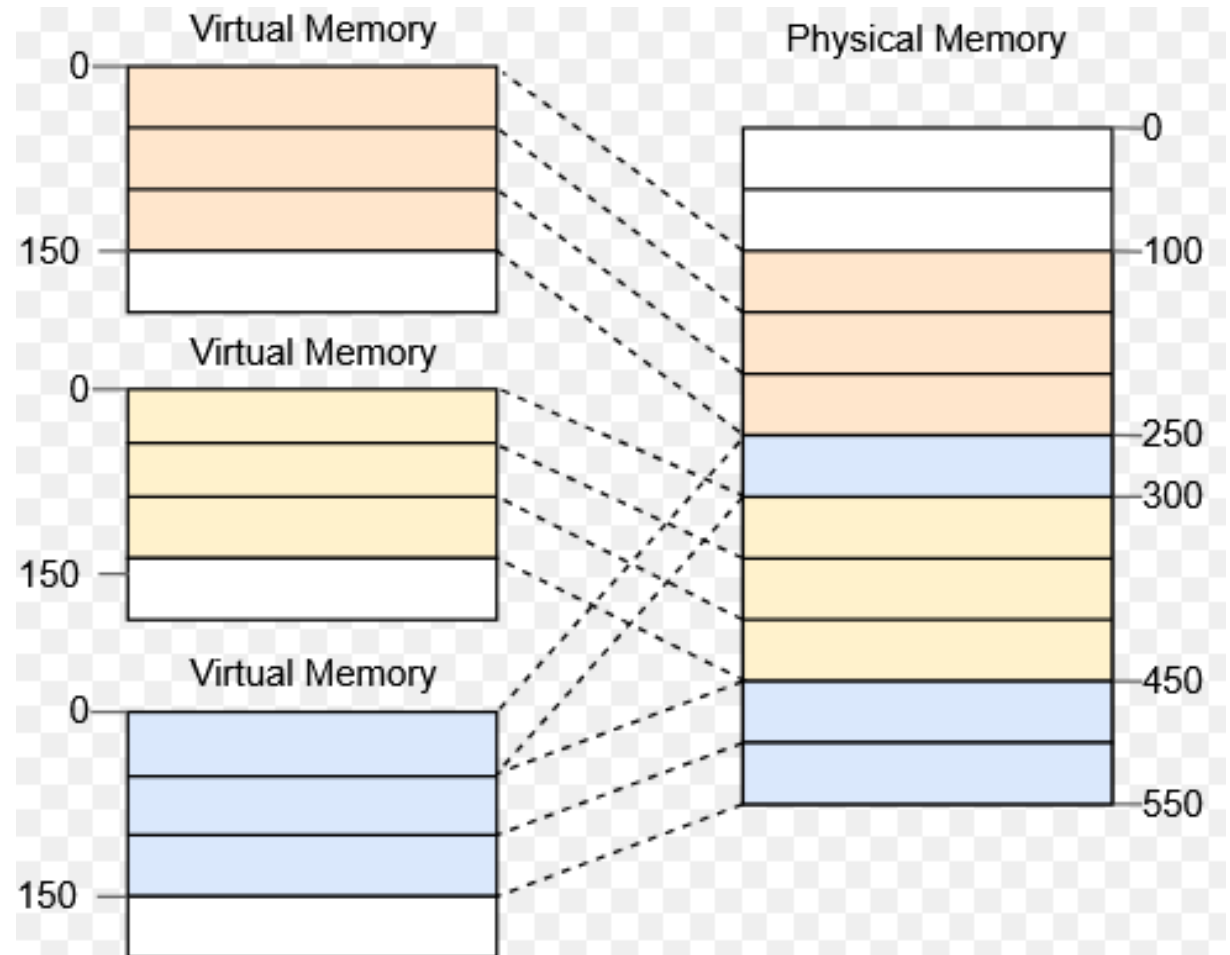
HOLES



PAGING?

Paging

- 프로세스의 메모리 자체를 잘게 쪼개서 실제 물리적 RAM에 넣어버림
- 이렇게 하면 Physical Memory의 작고 떨어진 공간까지 이용 가능
- OS에서 Logic Memory와 Virtual Memory는 거의 같은 뜻임



Paging

- 프로세스의 메모리를 자른 한 조각을 페이지라 하며, OS마다 다르지만 보통 일반적으로 4KB 크기임
- 같은 크기로 자른 RAM의 한 조각은 Frame이라 부름
- 페이지의 크기가 너무 작으면 조각 수가 많아져서 제어가 힘들고, 너무 크면 Fragmentation이 심해질 수 있음

PROCESS



PHYSICAL MEMORY

